HTBLuVA Graz-Gösting
Ibererstraße 15-21
8051 Graz

Abteilung für
Elektronik und Technische Informatik

DIPLOMARBEIT

IRIS - Infant Recognition and Intelligence System

Ausgeführt im Schuljahr 2025/26 von:

Linda Kalcher [K] 5AHEL
Katrin Tartler [T] 5AHEL

Betreut von:

Prof. DI DI Michelitsch Stephan, BEd, Bsc

Vorgelegt am

Kurzbeschreibung

Mithilfe unseres Systems soll es Eltern möglich gemacht werden, einen Gesamtüberblick über bestimmte Vitalwerte ihres Kindes zu erhalten. Dafür wurde ein Reisebabybett mit verschiedenen Sensoren ausgestattet, die Bewegung und Geräusche des Kindes messen. Die Daten werden ausgewertet und sind online über eine Webanwendung (Progressive Web App, PWA) einsehbar. In erweiterter Form wollen wir mit der Datenerfassung eine Grundlage für die Forschung an plötzlichem Kindstod bereitstellen. Um die Funktionsfähigkeit unseres Gerätes zu testen, wurde ein Testroboter gebaut welcher Babyverhalten simulieren soll. Eine umgebaute Puppe mit bewegenden Gliedmaßen, Geräuschwiedergabe und Wärmesimulation wurde entwickelt.

Abstract

Our system is designed to give parents the opportunity to get a compact overview of certain health data concerning their infant. For this purpose, a travel cot was equipped with various sensors that measure movement and noise. The data gets evaluated and can be looked up online via a Progressive Web App (PWA). In its expanded form, we aim to use the data collection to provide a foundation for research into sudden infant death syndrome (SIDS). To test the functionality of our system a testrobot to simulate baby behavior was built. A modified doll with moving limbs and heating simulation, which can also imitate babysounds was developed.

Eidesstattliche Erklärung

Danksagung

Inhaltsverzeichnis

1 Theorie **1**
1.1 Wie kam es dazu? . . . 1
1.2 Plötzlicher Kindstod . . . 1
1.3 Theorie zum allgemeineren System . . . 2
1.4 Was gibt es schon? . . . 2
1.4.1 Markt . . . 2
1.4.2 Forschung . . . 2
1.5 Babybettchen . . . 2
1.5.1 Overview . . . 2
1.5.2 Drucksensorik . . . 2
1.5.3 Wärmesensorik . . . 2
1.5.4 Audiosensorik . . . 3
1.5.5 PWA und alles was da dabei is . . . 3
1.6 Testroboter . . . 4
1.6.1 BLE und Interface . . . 4
1.6.2 Bewegungssimulation . . . 9
1.6.3 Geräuschsimulation . . . 11
1.6.4 Wärmesimulation . . . 11
1.7 3D Druck . . . 12
1.8 Platinendesign/ Platinenfertigung . . . 12
1.8.1 KiCad . . . 12
1.8.2 Ätzen . . . 13

2 Methodik **14**
2.1 Drucksensorik testen . . . 14
2.2 Wärmesensorik testen . . . 14
2.3 Geräuschsensorik . . . 15
2.4 Code (PWA) . . . 15
2.5 ESP für PWA . . . 15
2.6 Code Testroboter . . . 15
2.6.1 Client Code . . . 15

2.6.2 Server Code 21
2.7 HF sicher layouten 23
2.8 Motoren Switch 23
2.9 3D Konstruktionen 23
2.10 Main PCB 23
2.11 Testroboter PCBs 23

3 Ergebnis **24**

4 Diskussion **25**
4.1 weiter Sensoren 25
4.1.1 Temperaturmessung mit Körperkontakt 25
4.2 Spannungsversorgung Testroboter 25

Abbildungsverzeichnis

1.1 Microcontroller zu system-on-chip [8] 4
1.2 BLE Architektur [13] 5
1.3 BLE GAP [13] 6
1.4 GAP Verbindungsaufbau 7
1.5 Servo Motor Aufbau [19] 10

2.1 Verschaltung der Wärmekomponenten 15
2.2 Oberflächliche Beschreibung des Testroboter Codes 16
2.3 GUI des Testroboters 16

Listings

2.1 Tabellen Layout Interface 17
2.2 Buttons implemetieren 17
2.3 Beispielcode, Falscher Ansatz 18
2.4 Module 18
2.5 MAC, CHAR_UUID und BLE Thread 18
2.6 Funktion zur Verbindung mit ESP über BLE 19
2.7 Erstellen und Starten des BLE Threads 19
2.8 Kommandos über BLE Schnittstelle versenden 19
2.9 Beispiel einer Callback-Funktionen der Buttons 20
2.10 Zusatzbibliotheken 21
2.11 BLE Setup 22

1 Theorie

1.1 Wie kam es dazu?[K]

=> Linda

1.2 Plötzlicher Kindstod[T]

Unter dem Begriff Plötzlicher Kindstod versteht man das Versterben eines Babys welches das erste Lebensjahr noch nicht vollendet hat und unter keinerlei Vorerkrankungen litt. Im Zuge der Diagnose muss auch der Fundort des leblosen Kindes untersucht werden um einen Unfalltod, durch Ersticken an einer Decke, Kuscheltiers, etc. auszuschließen. Deshalb handelt es sich hierbei auch um eine Ausschlussdiagnose. In Österreich und anderen Industrienationen der Welt ist SIDS eine der häufigsten diagnostizierten Todesursachen.[24] Zudem belegen Statistiken, dass der Plötzliche Kindstod vor allem in den Wintermonaten auftritt und meist im Schlaf. [4] [30]

1.3 Theorie zum allgemeineren System

1.4 Was gibt es schon?

1.4.1 Markt

1.4.2 Forschung

1.5 Babybettchen

1.5.1 Overview

1.5.2 Drucksensorik[K]

1.5.3 Wärmesensorik[T]

Für die Überwachung von Wärmeparametern in dem Babybett kommen verschiedene Methoden in Frage. Der für unsere Anwendung wichtigste Bereich des Temperatur liegt, im Bereich der Körpertemperatur. Vor allem der Aspekt einer passablen Erkennung von Fieber bzw. Unterkühlung. Damit bewegen wir uns in einem Temperaturbereich von 30 bis 45 Grad Celsius. Da der menschliche Körper ein perfekt abgestimmtes System ist, ist der ideal Temperaturbereich äußerst gering. Von einer Unterkühlung spricht man ab einer Körpertemperatur von unter 34° C [11], von Fieber bei Säuglingen in den ersten Monaten schon ab 38°C [7] Für unsere Anwendung eines wertesammelnden Babybettes kommen zwei Möglichkeiten in Frage, eine Befestigung der Sensorik an der Matratze (siehe 4.1.1), für die Messung mit Körperkontakt, oder eine optische Messung über eine Wärmebildkamera.

Wärmebildkamera

Mit der Wärmebildkamera ist es möglich eine kontaktlos eine Übersicht der Temperatur zu bekommen. Die Wärmebildkamera empfängt die emmitierte Infrarotstrahlung, und gibt diese als bildliche Darstellung des gemessenen Objekts wieder. Es ist nicht möglich die menschliche Kerntemperatur, welche für eine Fieber oder Unterkühlungsangabe notwendig wäre zu ermittel, da die Außen- bzw. Hauttemperatur oft nicht ident zu dieser ist. Deshalb kann man keine genau Angabe im Bezug auf Fieber bzw. Unterkühlung geben, aber die gesammelten Daten können als Indizien gewertet werden. Es ist jedoch möglich eine etwas aussagekräftigere Messung zu erziehlen, wenn man sich Körperstellen ansieht welche die Kerntemperatur sehr gut oberflächlich wiederspiegeln. Ein Beispiel dafür sind die Augeninnenwinkel. [21]

1.5.4 Audiosensorik[T]

1.5.5 PWA und alles was da dabei is

1.6 Testroboter[T]

1.6.1 BLE und Interface

ESP32

Der ESP32 ist das zentrale Streuerungselement des Testroboters. Damit wird die Bewegungs-, Geräusch- und Wärmesimulation angesteuert. Der ESP32 ist ein system-on-chip (SOC) Microcontroller. Das bedeutet er umfasst viele Funktionen wie CPU, RAM, Microcontroller, und unterstützender Hardware, aufgebaut in einem Dev-Board. Wir benutzten den ESP32 DevKit C4

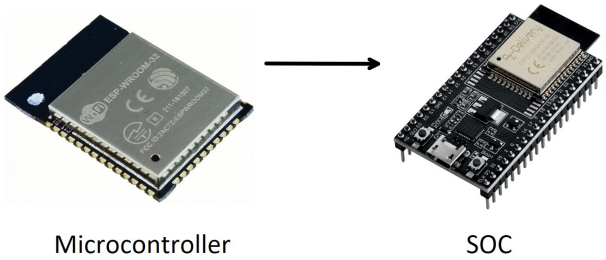

Abbildung 1.1: Microcontroller zu system-on-chip [8]

Damit der Testroboter für die entsprechenden Tests des Bettchens benutzt werden kann, wird mithilfe von BLE 1.6.1 vom ESP32 zu einem Laptop eine Verbindung hergestellt. Dort kann man zu Testzwecken die verschiedenenen Simulationsparameter in beliebigen Sequenzen ansteueren. [8]

BLE Architektur

Bluetooth Low Energy ist eine Bluetooth Protokoll, das im Vergleich zu Classic Bluetooth einen geringeren Stromverbrauch hat und somit auch einen niedrigere Datentransferrate erreicht. Es wird vor allem im Bereich Internet of Things (IoT) und für Kommunikation mit Messsensoren benutzt. BLE ist nicht kompatibel mit Classic Bluetooth. Das BLE Protokoll ist in drei Schichten geteilt.[14] siehe Abbildung 1.2

- Application Layer
- Host Layer
- Controller Layer

In der Application Layer können wir unsere Anwendung mit BLE realisieren. Hier implementiert man seine Anforderungen. In der Host Layer findet die Kommunikation zwischen Benutzeranwendung und Controllinterface der Hardware statt, man nennt

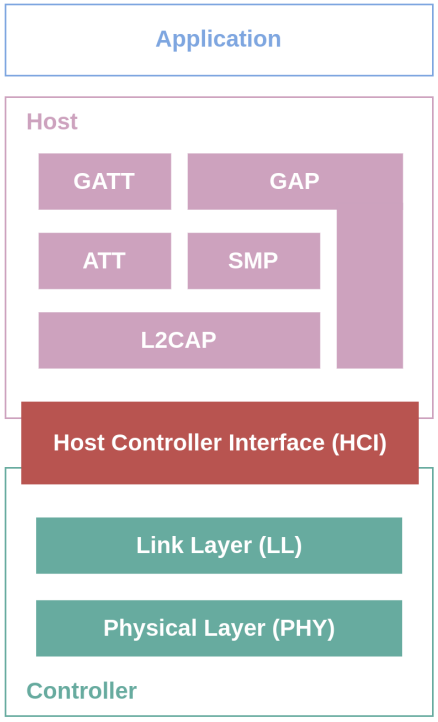

Abbildung 1.2: BLE Architektur [13]

diese Ebene auch Host Stack. Der in Kombination mit dem Controller Stack den Bluetooth Stack bildet. Alle relevanten Bluetooth LE Protokolle der Hostseite sind hier implementiert. In der letzten Schicht, der Controller Layer finden die physical und die link Layer ihren Platz. Sie interagieren direkt mit der Hardware. Die BLE-Protokolle der Host Schicht sind GAP, ATT, GATT, SMP und L2CAP.[13][14]

GAP (Generic Accsess Profile)

Ist für die Verbindung von zwei BLE-Geräten verantwortlich. Das Protokoll ist in 3 verschiedene Modi geteilt je nachdem welcher Verbindungsgrad erfüllt ist. Innerhalb dieser gibt es Rollen die den Geräten zugeteilt sind. Dies wird in Abbildung 1.3 veranschaulicht.[13]

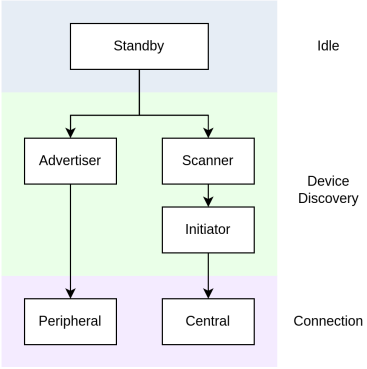

Abbildung 1.3: BLE GAP [13]

In unserem Projekt übernimmt der ESP32 die Rolle des Servers, und der Laptop ist der Client. Im GAP werden diese Rollen mit Peripheral (Server) und Central (Client) bezeichnet. [13] Anhand von Abbildung 1.4 lässt sich der Verbindungsaufbau und die unterschiedlichen Rollen welche die Teilnehmener dabei annehmen nachvollziehen.

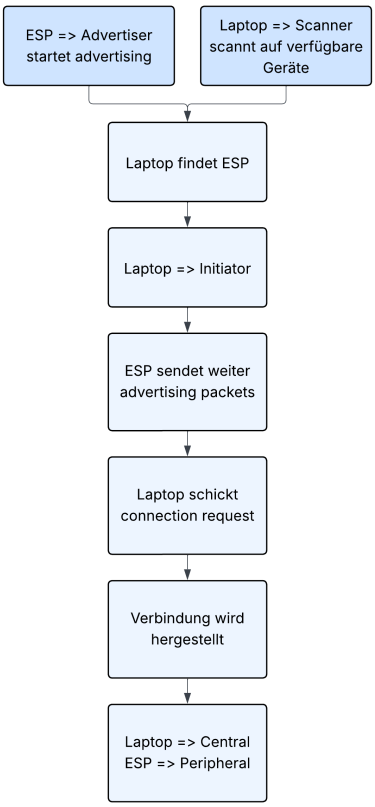

Abbildung 1.4: GAP Verbindungsaufbau

GATT (Generic Attribute Profile)

Dieses Protokoll basiert auf einer hierarchischen Einteilung von Characteristic, Service und Profile. Hier legt der Benutzter eine Anwendung anhand der vorgegebenen Struktur von BLE an. Diese wird in der ATT Layer technisch umgesetzt und gespeichert. siehe 1.6.1.
Characteristics sind Datenstrukturen die mit Attributen aufgebaut sind. Sie müssen ein Characteristic Declaration Attribute und ein Characteristic Value Attribute beinhalten. Optional gibt es die Möglichkeit noch weitere Descriptor Attributes in einer Characteristic zu implementieren.[13] Services sind ebenfalls mit Attributen aufgebaut. Ein sogennates Service Declaration Attribute muss definiert sein. In einem Service sind mehrere Characteristics mit einer ähnlichen Funktion oder zusammenhängenden Funktionen gesammelt. Sie agieren also nur als Container, und beeinhalten keine Nutzdaten. Services werden mit der sogenannten Service UUID (Universal Unique IDentifier) identiviziert. Bei der Wahl dieser kann man auf vordefinerte Service UUIDs zurückgreifen, welche von der Bluetooth Special Interest Group (Bluetooth SIG) zusammengefasst wurden. Die Bluetooth SIG stellt eine standardisierte Liste an Services für verschiedenste Anwendungen bereit. Diese beeinhalten zum Beispiel ein Service zur Geräteinformation,

einen speziellen Battery Service oder einen Heart Rate Measurement Service. Während Benutzerdefinierte Service UUIDs 128bit lang sein müssen sind standardisierte Service UUIDs oft nur 8-16bit lang. [6] [13] Profiles sind die übergeordnetesten Instanzen. Sie müssen gewisse Services haben muss um einen Standard zu erfüllen. Bei der Verwendung von BLE werden sie im Programmcode nicht definiert, wie Characteristics und Services. Ein Profile entsteht in dieser Hinsicht in einem Projekt nur konzeptionell. [13]

ATT (Attribute Protocol)

Auf dieser Ebene passiert die technische Umsetzung des GATT. Eine dort erzeugt Charakteristik wird am Server Device als Attribute gespeichert. Attribute sind strukturiert in Type, Value und Permission. Dabei werden sie im Speicher meist als Arrays angelegt-Attribute Table. Der Type eines Attributs ist ein 32- oder öfter benutzte 128bit langer UUID (Universally Unique IDentifier). [13]

L2CAP (Logic Link Control and Adaption Layer Protocol)

L2CAP ist für die Datenpaketverwaltung zuständig. Es wird auf Client- sowie auf Hostseite benutzt. Zu den Aufgaben gehören unter anderem Multiplexing, Segementierung und Wiederzusammensetzung der Daten. [32]

SMP (Security Manager Protocol)

Ist das Sicherheitsprotokoll von BLE, es ist ein Untermodul von GAP. [16]

NimBLE

NimBLE ist neben Bluedroid eines der von Espressif empfohlenen Bluetooth APIs (Application Programming Interface). Diese dienen den Programmieren als Schnittstellen, um mit der Bluetoothhardware interagieren zu können. Die Bezeichnung NimBLE spielt auf das englische Wort „nimble“ (flink, effizient) an und verrät so schon einiges über seine Eigenschaften. Im Vergleich zu Bluedroid ist NimBLE eine kompaktere Anwendung, welche weniger Speichernutzung und einfacheren Code unterstützt. Somit ist NimBLE ressourcenschonender und effizienter. Es ist nur für BLE geeignet und kann nicht zur Kommunikation mit Classic Bluetooth benutzt werden. [12] [1]

Bleak

Bleak ist ein ist ein Python API die es ermöglicht mit BLE-Geräten in Verbindung zu treten und zu kommunizieren. Auch hier ist der Name Programm. Bleak steht

für Bluetooth Low Energy Platform Agnostik Klient. Das bedeutet das egal welche Plattform oder Betriebssystem genutzt wird, die Anwendung funktioniert flexibel. [20]

Dear PyGui

Mit der Python Bibliothek Dear PyGui (DPG) lassen sich einfache GUI (Graphical User Interface) Anforderungen in Python realisieren. Man nutzt sie zur Erstellung von Diagrammen, Tabellen und einfachen bis komplexen User Interfaces. DPG ist dynamisch, GPU-beschleunigt und sehr anwenderfreundlich. [22]

1.6.2 Bewegungssimulation

Bewegungsabläufe

In einer Studie der Segawa Neurological Clinic for Children aus dem Jahr 1980 sammelten die Forschenden Daten über das Bewegungsverhalten von 25 Säuglingen. Mithilfe EKG Messungen und visuelen Beobachtungen kategoriesierten sie die Bewegungen der Kinder in drei Gruppen. [17]

- GM(gross movements)

  Die größten Bewegungen, ganze Gliedmaßen werden länger als 0,5s bewegt.

- LM(localized movements)

  Diese etwas kleineren Bewegungen dauerten auch länger als 0,5s.

- TM(twitch movements)

  Kleine Muskelzuckungen die weniger als 0,5s andauerten.

Mit der Zeit nimmt die Anzahl an Bewegungen in einer Nacht immer weiter ab. Angefagen bei dem zuckenden TM Bewegungen bis hin zu den GMs. Diese Bewegungsabläufe sind eng mit der motorischen Entwicklung eines Kindes und der generellen Entwicklung des Zentralennervensystems verbunden. Die Forscher sehen darin einen Methode um neurologische Auffälligkeiten bei Säuglingen frühest möglich zu erkennen. [17]

Servomotoren

Ein Servomotor ist ein Elektromotor welcher über eine interne Sensorik verfügt, die zur Regelung der Position, Geschwindigkeit oder des Drehmoments verwendet wird. Der Motor besteht aus vier Hauptkomponenten, die auch in Abb 1.5 (bezogen auf unsere verwendeten Servos, es könnte auch ein AC-Motor, oder andere Senorik verwendet werden) veranschaulicht sind sind. [33] [19]

- DC-Motor => sorgt für den Antrieb
- Getriebe => reduziert Drehzahl und erhöht Drehmoment
- Positionsmessung => Messkomponente der Reglung
- Steuerplatine => regelt den Motor nach

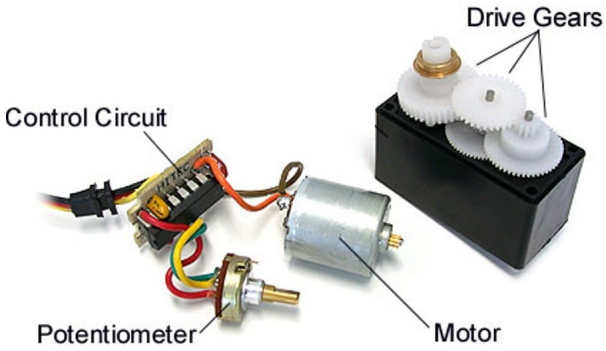

Abbildung 1.5: Servo Motor Aufbau [19]

Die Kombination dieser einzelnen Komponenten erzeugt die typische Servofunktion. Der DC-Motor funktioniert durch das physikalische Prinzip der Lorentzkraft. Die Kraft die ein Magnetfeld auf bewegte elekrische Ladungen (z.B. Elektronen) ausübt. Aufgebaut wird der Motor mittels Stator-Rotor-Prinzip. Der Stator (ein Magnet) ist nicht in Bewegung und erzeugt ein konstantes Magnetfeld. Der Rotor (eine Spule), auch Anker genannt muss drehbar montiert sein. Wird der Motor nun mit Spannung versetzt, fließt Strom über die sogennaten Bürsten, zu den Spulen. So beginnt der Anker zu rotieren, getrieben durch die Lorentzkraft. Das funktioniert nur bis zu einer gewissen Position. Ist der Anker dort angelangt, kommt der Kommutator ins Spiel. Er ändert die Phase des Stroms um 180°, dadurch bleibt die Drehbewegung kontinuierlich erhalten. Das Potentiometer und die Steuerplatine sorgen für die Regelung des Servomotors. Das Potentiometer erfasst die momentane Position (in Form eines Winkels) und gibt diese an die Regelung der Platine weiter. Diese hat durch einen Microcontroller (z.B. ESP32) einen Sollwert erhalten und regelt den Servo genau zu diesem hin. Die Ansteuerung des Servomotors durch den Microcontroller erfolgt mittels PWM (Pulsweitenmodulation, engl. pulse width modulation). Die Logik des Servos bekommt den entsprechenden Winkelwert nicht durch eine bestimmte Spannung sondern durch die Länge eines Pulses. Diese Länge berechnet der ESP je nach gewünschtem Winkel und sendet sie über PWM-Pins an die Servomotoren. [33] [19]

1.6.3 Geräuschsimulation

DFPlayer Module

Das DFPlayer Module von Az-Delivery ist ein eigenständiger Audio-Decoder, der die Wiedergabe von MP3- und WAV-Dateien von einer micro SD-Karte ermöglicht. Es lässt sich über eine UART-Schnittstelle oder manuell über eine Taststeuerung der digitalen Eingänge ansteuern. [9]

UART Kommunikation

Für die UART (Universal Asynchronous Receiver/Transmitter) Kommunikation werden die Daten bitweise über eine Leitung gesendet. Asynchron bedeutet es gibt kein gemeinsames Taktsignal, baudrate von Modul und ESP32 müssen jedoch die gleiche sein, standardgemäß 9600. Die Daten werden in festen Formaten gesendet die durch Start- und Stoppbit abgerenzt werden. Aus dem Datenblatt des DFPlayer Moduls kann man die commands für die einzelnen Funktionen des Moduls herauslesen. Zur Programmierung ist es nicht erforderlich die commands manuell zu codieren da die Library DFRobotDFPlayerMini [10] diese in Funktionen implementiert hat.

Lautsprecher

Die Audioausgabe erfolgt über einen 8 Ohm Lautsprecher. Dieser wandelt die eletrischen Audio Signale in mechanische (Schallwellen) um die den Ton übertragen.

1.6.4 Wärmesimulation

Keramikwiderstände

Keramikwiderstände sind im Vergleich zu herkömmlichen $\frac{1}{4}$-Watt Widerständen mechanisch und thermisch viel stabiler. Sie werden überall dort eingesetzt, wo starke Verlustleistungen auftreten, da sie oft ein Vielfaches der Leistung aushalten. Dank ihrer Keramikhüllen dienen sie als gute elektrische Isolatoren und helfen die Wärme besser in die Umgebung abzugeben. Dieser Aspekt führt dazu, dass sie in bestimmten Fällen gezielt als Wärmequellen eingesetzt werden können. Im Vergleich zu anderen Widerstandstypen, die auch oft Keramikkomponenten aufweisen, werden Keramikwiderstände durch das Vermischen von Kohlenstoffpulver (elektrischer Leiter) und Keramikmaterial (Isolator) gefertigt. Dabei bestimmt das Verhältnis den Widerstandswert, befindet sich viel Kohlenstoffpartikel in der Mischung wird der Widerstand geringer und umgekehrt. Die fertige Masse wird in Form gepresst, oft mit einer zusätzlichen Schicht Keramik überzogen und im Ofen gebrannt. [27] [26]

PT100

Der PT100 ist ein temperaturabhängigier Widerstand der Kategorie der PTCs (Positive Temperature Coefficient). Temperaturabhängige Widerstände verändern ihren Widerstandswert je nach gegebener Umgebungstemperatur. Ein positiver Temperaturkoeffizient bedeutet, dass der Widerstand bei steigender Temperatur auch steigt. Pro Grad ändert sich der Widerstandswert des PT100 um 0,385Ω [18]. Den PT100 zeichnet aus, dass er bei einer Temperatur von 0 Grad eine Widerstand von 100 Ω hat. Aufgrund dieser physikalischen berechenbarkeit eignet sich der PT100 perfekt als Messkomponente zur Bestimmung von Temperturwerten.

1.7 3D Druck[T]

Der 3D Druck ermöglicht es nahezu jede erwünschte Konstruktion zu realisieren. Der Weg bishin zu selbst gefertigten Gehäuseteilen beginnt mit dem Designen der Konstruktion. Dazu eigent sich Fusion360 besonders, da wir als Schüler einen kostenlosen Zugang erhalten können. Eine fertige Konstruktion wird mithilfe einer Slicer-Software in für den 3D-Drucker lesbare Schichten kodiert. Der Drucker kann dann Stück für Stück jede einzelne Schicht nach oben auftragen und man erhält so ein fertiges Objekt. In unserem Fall war diese Technik besonders beim Zusammenbau des Testroboters gefagt. Motorbefestigungen, Gelenksplatten und eine stablilisiernede Innenkonstruktion wurden mithilfe dieses Verfahrens angefertigt siehe 2.9.

1.8 Platinendesign/ Platinenfertigung

1.8.1 KiCad[T]

KiCad ist kostenloses open-source Schaltplan- und Layoutdesigntool. Es kann betriebsystemübergreifend genutzt werden. KiCad ermöglicht eine Schritt für Schritt Projektentwicklung, vom Einbeziehen der individuellen Bauteile bis zum 3D-modellierten Abbild der erstellten Platine. Es gibt viele verschieden Plugins die kombiniert wertvolle Features ergeben. Ein besonderer Vorteil ist die große Sammlung an Symbolen, Footprints oder 3D-Modellen und das einfache Konfigurieren oder Einbeziehen individueller Komponenten. Für die weitere Fertigung der Platine lassen sich ganz leicht, benötigte Gerber und Bohrdateien exportieren. [23]

1.8.2 Ätzen

Linda Part

2 Methodik

2.1 Drucksensorik testen[K]

2.2 Wärmesensorik testen[T]

Für die Simulation der Temperatur des Kindes, wurden im Testroboter Keramikwiderstände eingebaut, welche als Wärmequellen genutzt werden. Dazu wurde der Aufbau und die Verschaltung mittels Steckbrett getestet. Besonders zu beachten ist die hohe Leistung die für eine Wärmesimulation nötig ist. Dementsprechend ist ein Test auf dem Steckbrett schwierig, da diese meisten nicht für hohe Leistung ausgelegt sind. Weiters ist auch die Vesorgung ein Problem, weil die Leistung über eine hohe Stromstärke erbracht werden muss.

$$P = I^2 \cdot R$$
$$P = \frac{U^2}{R} \tag{2.1}$$

Die verwendeten Keramikwiderstände haben einen Widerstandswert von 2,2Ω. Die Verschaltung erfolgt mit 2 in Serie geschalteten Strängen mit jeweils 3 parallel geschalteten Keramikwiderständen. Siehe Abbildung 2.1, einem Ausschnitt des Schaltplans der Platine aus 2.11. Somit ergibt sich ein Gesamtwiderstand von ca. 1,5Ω. Als Spannungswert werden 7,4V eingesetzt. Diese entsprechen dem Spannungswert eines 2 Zellen Lithium-Ionen Akkus. Somit ist ein Nachrüsten auf eine eigenständige Spannungsversorgung möglich siehe 4.2.

$$P = \frac{7,4^2}{1,5} = 36,507W$$
$$I = \sqrt{\frac{36,507}{1,5}} = 5,0004A \tag{2.2}$$

Der benötigte Strombedarf liegt bei unserer Leistung von 36,507W also bei ca 5A. Dieser Bedarf wird von einem 20A Labornetzteil gedeckt. Aufgrund der Wahl dieser Spannungsversorgung wurde beschlossen die weiteren Komponenten des Testroboters

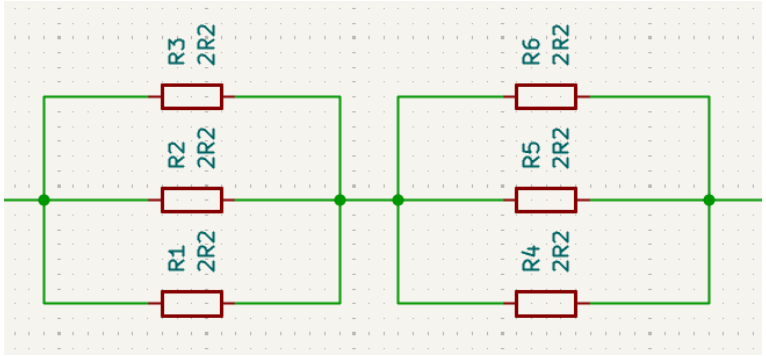

Abbildung 2.1: Verschaltung der Wärmekomponenten

ebenfalls mit dem Netzteil zu betreiben.

2.3 Geräuschsensorik[T]

2.4 Code (PWA)[K]

2.5 ESP für PWA[K]

2.6 Code Testroboter[T]

Der Code zur Steuerung des Testroboters besteht aus zwei pythonbasierten Programmen siehe 2.6.1. Welche die Darstellung das Graphical User Interface (gui.py) und die BLE Backend Kommunikation (ble_backend.py) implementieren. Damit werden die Client-Funktionen des Laptops abgedeckt. Der Code für den ESP32, siehe 2.6.2 muss die Funktionen der Serverrolle im BLE Aufbau erfüllen. Also gesendete Commands müssen eine Funktion starten. Der Aufbau der Code Architektur des Testroboters ist in Abbildung 2.2 veranschaulicht.

2.6.1 Client Code

Interface Darstellung

Das Interface ist vor allem für den Testbetrieb von großer Wichtigkeit, es wird benutzt um die einzelnen Simulationsparameter anzusteuern. Für den Aufbau des Interfaces wird die Pythonlibray Dear PyGui (DPG) verwendet. Siehe auch 1.6.1 Damit DGP ein Fenster erzuegen kann, müssen 6 Funktionen unbedingt im Programmcode enthalten sein.

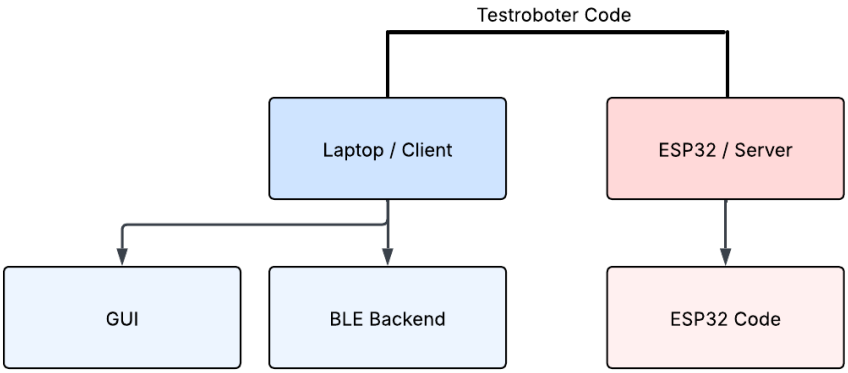

Abbildung 2.2: Oberflächliche Beschreibung des Testroboter Codes

- createcontext()
- createviewpoint()
- setupdearpygui()
- showviewpoint()
- startdearpygui()
- destroycontext()

Wird eine dieser Funktionen nicht korrekt aufgerufen, führt das zu Problemen, weil das Interface nicht richtig durchlaufen kann. Die Initialisierung, Darstellung und Interaktion des GUI könnten sonst nicht realisiert werden. [22] Innerhalb dieser Funktionen wird das Interface dann für die eigenen Anforderungen aufgebaut.

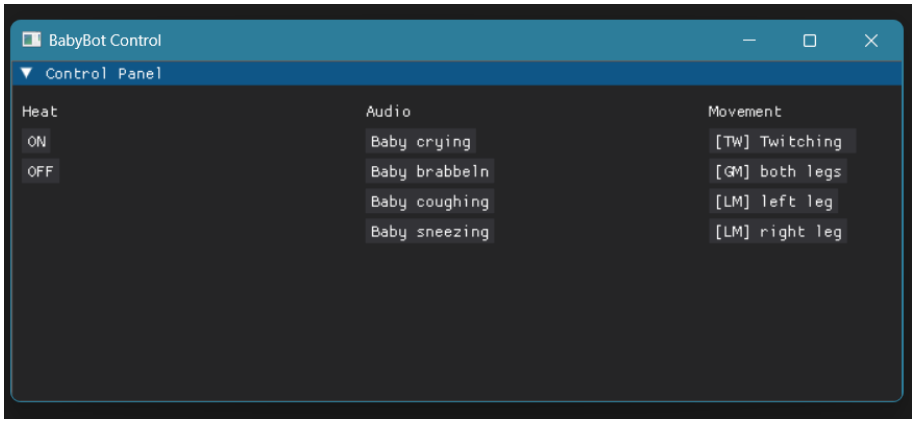

Abbildung 2.3: GUI des Testroboters

Der Aufbau des Graphischen Interfaces ist primär mit Buttons ausgestattet die bei einem Klick, eine Callback Funktion ausführen. Es gibt eine Spalte für jeden ansteuerbaren Parameter, siehe Abbildung 2.3 Die Buttons in der Wärmespalte, triggern das Ausschalten bzw. Einschalten der Wärmequellen. In der nächsten Spalte ist die Bewegungssteuerun abgebildet, hier kann man aus verschiedenen Bewegungssequenze auswählen. Die Sequenzen sind an die beschriebenen Bewegungsmuster aus 1.6.2 angelehnt. In der Audio-Spalte gibt es Buttons welche unterschiedliche Babygeräusche triggern. Es gibt die Auswahl zwischen weinen, brabbeln, husten und niesen. Diese 30 Sekunden Audioinhalte wurden KI genriert. Für das Erstellen des Spalten Layouts werden bestimmte Funktionen angewendet, siehe Listing 2.1

Listing 2.1: Tabellen Layout Interface

```
10 with dpg.table(header_row = False):
11     dpg.addtablecolumn();
12     dpg.addtablecolumn();
13     dpg.addtablecolumn();
14 
15     with dpg.table_row():
16          with dpg.group():
```

In der "Gruppe"(group) die in Zeile 16 implementiert ist, werden nun die Buttons für eine Spalte und somit für einen bestimmten Simulationsparameter erstellt. Zu sehen am Beispiel für die Wärmesimulation in Listing 2.2. Danach noch eine Gruppe für Bewegungsknöpfe und eine für Audioknöpfe.

Listing 2.2: Buttons implemetieren

```
16 with dpg.group():
17     dpg.add_text("Heat")
18     dpg.add_button(label="ON", callback = ble_backend.heat_on)
19     dpg.add_button(label= "OFF", callback = ble_backend.
          heat_off)
```

Das erste was dem Button übergeben wird ist ein label, dies ist nicht zwingend erforderlich, aber für die Verwendung von einem Testinterface natürlich sehr von Vorteil. Damit kann man wissen welche Aktion der Button ausführt. Der zweite Parameter ist die Callback-Funktion, erst mit dieser kann der Button bei Klick auch eine Aktion auslösen. Da die Buttons des GUIs über BLE verschiedene Kommandos an den ESP im Testroboter schicken (welcher dann die genaue Ansteuerung der Komponenten übernimmt) sind die Callback-Funktionenen im zweiten Programm in der Client Architektur implementiert, in der Datei ble_backend.

Interface BLE Kommunikation

Die Kommunikation die per Button Klick gestartet wird is in der Datei ble_backend.py beschrieben. Der erste Ansatz war mit der Theorie verbunden, eine normale def callback-Funktion zu schreiben, welche eine await Funktion mit dem Schreiben des BLE Kommandos beinhalten sollte siehe Listing 2.3. Die await Funktion ist in Python ein Werkzeug um einen Task zu pausieren bis ein anderer beendet ist. Das schafft eine Art parallels System [31].

Listing 2.3: Beispielcode, Falscher Ansatz

```
#callback-funktion
def heat_on():
    await client.write_gatt_char(CHAR_UUID,b"heat")
```

Das ist nicht möglich weil ein await nur in einer asynchronen Funktion in Python benutzt werden darf. Die Callback-Funktion ist aber def. Daher ist es am Besten mit den Modulen threading und asyncio, als Ergänzung der verwendeten bleak Library zu arbeiten, siehe Listing 2.4.

Listing 2.4: Module

```
import threading
import asyncio
from bleak import BleakClient
```

Zu Beginn müssen die MAC-Adresse des ESP und die Charakter UUID, welche im ESP Code defniert sind, auch in der Client Architektur gespeichert werden. Der zweite Schritt ist mithilfe der Funktionen des Threading-Moduls einen eigenen Thread für BLE zu erstellen. So wird sicher gestellt, dass auch beim Senden eines Kommandos aus ble_backend.py gui.py weiterhin parallel dazu laufen kann. Zu sehen sind diese Anforderungen in Listing 2.5.

Listing 2.5: MAC, CHAR_UUID und BLE Thread

```
ESP_MAC = "a0:b7:65:69:9d:3e"
CHAR_UUID = "abcd1234-ab12-34cd-56ef-1234567890ab"
ble_loop = asyncio.new_event_loop()
```

asyncio.new_event_loop() erstellt uns eine Eventloop mit dem Namen ble_loop. Diese ist keine der Funktion der threading libraray sondern hilf uns bei der Anwendung der threading Funktionen für unseren Zweck. Als nächstes wird mit den Funktionen der bleak library eine Verbindungsfunktion erstellt, siehe Listing 2.6.

Listing 2.6: Funktion zur Verbindung mit ESP über BLE

```
9  async def ble_connect():
10     global client
11     client = BleakClient(ESP_MAC)
12     await client.connect()
13     print("client_connected")
```

Zuerst wird eine Variable client erstellt, diesem client wird mit der Funktion BleakClient(ESP_MAC), ein Gerät zugewiesen (über die MAC-Adresse) mit dem es Verbindung aufnehmen soll. Dementsprechend wird der Laptop im BLE Frequenzbreich nach diesem Gerät suchen und dann über das GAP-Protokoll(siehe 1.6.1) eine Verbindung aufbauen. Dieser Verbindungsaufbau wird über await client.connect() initialisiert.Das debugging comment am Ende ist optional.

Listing 2.7: Erstellen und Starten des BLE Threads

```
16 def ble_start_thread():
17     def run():
18         asyncio.set_event_loop(ble_loop)
19         ble_loop.run_until_complete(ble_connect())
20         print("BLEloop_running")
21         ble_loop.run_forever()
22
23     threading.Thread(target=run,daemon=True).start()
```

Die Funktion ble_start_thread (Listing 2.7) ist entscheident für die parallel laufenden Eigenschaften des Programms. Zuerst wird eine run() Funktion definiert welche dann dem Thread in Zeile 23 als Argument übergeben werden kann. Zuerst wird hier dem Term event_loop unsere zuvor erstellte ble_loop übergeben. Zusätzlich wird sichergestellt das ble_loop immer läuft. Mit der letzten Zeile der Funktion ble_start_thread wird der Thread zur BLE Kommunikation schließlich gestartet. Das nicht blockierende, parallele System ist ab diesem Punkt fertig initialisiert und gestartet. Ab diesem Moment ist es möglich die eigentliche Funktion (also das Senden der Kommandos) zu implementieren. Die Zeilen davor sind nur eine Absicherung damit das System fehlerfrei und sauber laufen kann. Für das Schicken der Kommandos ist die Funktion ble_send_cmds verantwortlich. (Listing 2.8)

Listing 2.8: Kommandos über BLE Schnittstelle versenden

```
25 async def ble_send_cmds(data: bytes):
26     if not client.is_connected:
27         print("not_connected")
```

```
28            await client.connect()
29
30        await client.write_gatt_char(CHAR_UUID, data)
31        print(f"sent{data}cmd")
```

Zuerst folgt eine Absicherung auf eine aufrechte Verbindung zwischen ESP und Laptop (Zeilen 26-28). Für das Versenden eines Kommandos wird nur die Funktion client.write_gatt_char(CHAR_UUID, data) aus der bleak library benötigt. Hier wird das Kommando das als data Parameter übergeben werden muss, an die entsprechende Characteristic des Servers (ESPs) geschickt.

Die verschiedenen Kommdandos die die Callback-Funktionen der Buttons darstellen sind so definiert (Listing 2.9).

Listing 2.9: Beispiel einer Callback-Funktionen der Buttons

```
35 def heat_on():
36     asyncio.run_coroutine_threadsafe(ble_send_cmds(bytearray
           ([9])), ble_loop)
37 def heat_off():
38     asyncio.run_coroutine_threadsafe(ble_send_cmds(bytearray
           ([10])), ble_loop)
```

Zu sehen sind die Callback-Funktionen der Wärmesimulation die in der gui.py als Callbacks für die Heat Buttons aufgerufen werden, siehe (Listing 2.2). Diese Funktion wird also aufgerufen wenn im Interface jemand einen Heat Button drückt. Bei dem Button der die Wärmequelle einschalten soll, wird die Zahl 9, als bytearray übermittelt. Nach diesem Muster sind alles Callback-Funktionen generiert, sie haben nur entsprechend andere Zahlen die übermittelt werden. Der Grund für die Übertragung der Daten mit bytearrays ist, dass diese laut ATT Protokoll sowieso in Arrays von Oktets gesendet werden. Wenn man im Code gleiche solche definiert, gibt es keine unnötige hin und her Kovertierung, wie es bei zum Beispiel String Kommandos der Fall wäre. [5][20]

virtual enviornment

Für die Installation der Pakete Bleak und Dear PyGui wird ein virtual enviornment verwendet. Das virtual enviornment (venv) ist eine generiert Umgebung die bei der Entwicklung von eigenständiger Software viele Vorteile bringt. In unserem Fall ist es vor allem um ein sauberes System zu belassen. Die Pakete die wir für das Interface und die Kommunikation brauchen werden nicht global installiert, sondern sind nur in dem entsprechenden Projekt zu finden. In weiterer Folge wäre es möglich das GUI in eine Programmdatei unzuwandeln. Hier ist es auch von Vorteil mit einem venv zu arbeiten.

2.6.2 Server Code

Nachdem die Kommandos gesendet wurden ist es die Aufgabe des ESPs die entsprechenden Kompontenten anzusteuern. Für die Programmierung des ESP32 verwenden wir VS-Code mit der Extension PlatformIO. PlatformIO ist eine Entwicklungsumgebung welche die wesentlichen Schritte für das Programmieren eines Microcontrollers, wie Kompilieren, Bibliotheksverwaltung und Hochladen auf diesen in einer Umgebung zusammenfasst [28]. Die Anforderungen um ein Projekt zu erstellen sind die Auswahl eines gewünschten Frameworks und des Boards welches man verwendet. Der Framework beschreibt die Softwaregrundlage die für die Programmierung genutzt werden soll. Damit ist ein großer Teil der Struktur des Projekts konfiguriert [34]. Wird unser ESP als DevBoard eingestellt stehen zwei Frameworkoptionen zur Auswahl. Der Arduino Framework, welcher auch in der ArduinoIDE genutzt wird, oder der offizielle Framework von Espressif, ESP-IDF [29]. Der Framework Arduino stellt die Grundvoraussetzungen bereit, mit einer Optimierung auf ESP-IDF hat man zugriff auf eine genauer auf den Microcontroller abgestimmte Funktion, aber es ist ein viel komplexer [15]. In diesem Fall wird der Arduino Framework genutzt [2]. Die Aufgabe des Server Codes können in fünf unterschiedliche Sektionen geteilt werden. Drei davon entsprechen der Ansteuerung der drei Simulationsparameter, Wärme, Bewegung und Geräusch. Die vierte ist eine BLE Verbindung mit dem Laptop herzustellen und dem fünften Punkt einer parallel agierenden Umgebung um blockieren zu verhindern. Zu sehen in Listing 2.10 ist eine Auflistung der verwendeten Zusatzbibliotheken um die entsprechenden Funktionen zu erfüllen.

Listing 2.10: Zusatzbibliotheken

```
#include <SchedTask.h>
#include <SchedTaskT.h>
#include <ServoEasing.hpp>
#include <NimBLEDevice.h>
#include <DFRobotDFPlayerMini.h>
```

BLE Verbindung und Kommandoauswertung

Für die interaktion mit der Bluetoothhardware des ESP wird NimBLE (siehe 1.6.1) benutzt. Um die Kommunikation richtig aufzusetzten wird zuerst eine Art BLE Setup durchgeführt. Wie der Name schon vermuten lässt befindet sich diese Konfiguration im Setup, also dem Programmteil welcher immer nur einmal zu anfang ausgeführt wird, bzw. bei jedem neuen flashen oder neustarten [3]. Diese Konfiguration ist in Listing 2.11 abgebildet.

Listing 2.11: BLE Setup

```
126  NimBLEDevice::init("Robo");
127
128  NimBLEAddress addr = NimBLEDevice::getAddress();
129  Serial.print("BLE␣MAC:");
130  Serial.print(addr.toString().c_str());
131
132  NimBLEServer * server = NimBLEDevice::createServer();
133  NimBLEService * service = server->createService("
       12345678-1234-1234-1234-1234567890ab");
134
135   NimBLECharacteristic* characteristic =
136      service->createCharacteristic(
137          "abcd1234-ab12-34cd-56ef-1234567890ab",
138          NIMBLE_PROPERTY::READ | NIMBLE_PROPERTY::WRITE
139      );
140
141  characteristic->setCallbacks(new CommandCallback());
142  characteristic->setValue("ready");
143
144  service->start();
145
146  NimBLEAdvertising* adv = NimBLEDevice::getAdvertising();
147  adv->addServiceUUID(service->getUUID());
148  adv->start();
```

Die Sektion BLE Setup folgt dem allgemeinen Standard zur initialisierung eines BLE-Gerätes. Mit NimBLEDevice::init("Robo"), wird dem Gerät der von uns gewählte Name Robo zugewiesen. Das Auslesen der MAC-Adresse des ESPs in weiterer Folge ist um die richtige Adresse im ble_backend.py Code eintragen zu können, wie in Listing 2.5 angegeben. Danach wird (Zeile 128 und 129) auf GATT Ebene ein Server erstellt und diesem ein Service zugeschrieben. Die Erstellung von einem Service ist wichtig für die Funktion der BLE Verbindung (die Funktion eines Services ist in 1.6.1 nachzulesen), der Service ist mit 12345678-1234-1234-1234-1234567890ab, der Service_UUID identifizierbar. Dies ist eine benutzerdefnierte Service UUID die zufällig erstellt wurde. In diesem Service wird eine Charakteristik für das schreiben und lesen von Daten zwischen der Laptop-ESP Verbindung erstellt (Zeile 131 -135). Die CHAR UUID die hier angegeben ist muss auch im Client Code definiert sein, siehe 2.5. Mit characteristic->setCallbacks(new CommandCallback()) wir festgelegt was passieren soll wenn der

Client Nutzdaten schickt. Es wird eine Callback-Funktion defniert die ausgeführt werden soll wenn der Server eine Nachricht empfängt. characteristic->setValue("ready") ist eine Nachricht an den Client, sollte dieser die Characteristic lesen, empfängt er "ready". Mit der Funktion service->start() ist die Konfiguration des Services abgeschlossen, er ist nun auf GATT-Ebene registiert und gestartet. In den Zeile 142 bis 144 wir nun die GAP-Ebene angesprochen. Zu Beginn wir der GAP-Zugriff mit NimBLEAdvertising* adv = NimBLEDevice::getAdvertising() gestartet. Dann der Service für den Client dargestellt und dann beginnt der ESP zu werben (adverting), um die Verbindung mit dem Client einzugehen. Verbindungsaufbau in unserem Projekt auch in Abblidung (1.4) veranschaulicht.

SchedTask Libray und Multithreading

Beim Testen der Anwendungen sind öffter Fehler durch das Blockieren von verschiedenen Komponenten durch einzelne Tasks aufgetreten. War zum Beispiel ein Motorbewegungs-Task aktiv sind andere Untersysteme blockiert gewesen. Für eine Optimierung wurde die library SchedTask [25] verwendet.
Multithreading bietet eine angenehme Lösung zu diesem Problem. Beim Multithreading werden

ServoEasing Library

2.7 HF sicher layouten[T]

2.8 Motoren Switch[T]

2.9 3D Konstruktionen[T]

2.10 Main PCB[K]

2.11 Testroboter PCBs [T]

3 Ergebnis

4 Diskussion

4.1 weiter Sensoren

4.1.1 Temperaturmessung mit Körperkontakt[T]

Die Messung der Körperkerntemperatur ist für eine exakte Bestimmung von Fieber oder Unterkühlung essentiell. Dafür können NTC (Negative Temparature Coefficient) oder auch Halbleitersensoren mit integrierten ICs genutzt werden. Diese liefern oft direkt Ergrbnisse, die nicht mittels Software und berechnet werden müssen. Anders als bei einer Messung mittels NTC Widerständen, mit denen man mithilfe eines Spannungsteilers die Temperaturwerte aus dem jeweiligen Spannungswert berechnen muss.

4.2 Spannungsversorgung Testroboter[T]

Literatur

[1] Apache Software Foundation. *BLE User Guide*. Apache Software Foundation. URL: https://mynewt.apache.org/latest/network/index.html (besucht am 21. 03. 2026).

[2] Arduino. *Arduino Language Reference*. Arduino. 2024. URL: https://www.arduino.cc/reference/en/ (besucht am 02. 04. 2026).

[3] Arduino. *setup()*. Arduino. 2024. URL: https://docs.arduino.cc/language-reference/de/struktur/sketch/setup/ (besucht am 02. 04. 2026).

[4] Thomas Bajanowski und Christian Poets. „Der plötzliche Säuglingstod: Epidemiologie, Ätiologie, Pathophysiologie und Differenzialdiagnostik“. In: *Deutsches Ärzteblatt* 101.47 (Okt. 2004), S. 6815. eprint: https://www.aerzteblatt.de/archiv/pdf/6e55d777-a594-4b62-adaf-e1a9486b428d.

[5] Bluetooth Special Interest Group. *Bluetooth Core Specification Version 5.4 - Attribute Protocol (ATT)*. Attribute value - 3.2.3. 2023. URL: https://www.bluetooth.com/specifications/specs/core-specification/ (besucht am 01. 04. 2026).

[6] Bluetooth Special Interest Group. *GATT Assigned Numbers*. Bluetooth Special Interest Group. 2024. URL: https://www.bluetooth.com/specifications/assigned-numbers/ (besucht am 02. 04. 2026).

[7] Bundeszentrale für gesundheitliche Aufklärung (BZgA). *Fieber beim Kind*. 2026. URL: https://www.kindergesundheit-info.de/themen/krankes-kind/krankheitszeichen/fieber/ (besucht am 01. 04. 2026).

[8] AZ-Delivery Vertriebs GmbH. *ESP-32 DevKit C V4 Datenblatt*. 2021. URL: https://cdn.shopify.com/s/files/1/1509/1638/files/ESP-32_DevKit_C_V4_Datenblatt_AZ-Delivery_Vertriebs_GmbH_24ec770f-c65e-4bd3-92c9-cd64b4d070b8.pdf.

[9] AZ-Delivery Vertriebs GmbH. *MP3 Player Module Datenblatt*. URL: https://cdn.shopify.com/s/files/1/1509/1638/files/MP3_Player_Modul_Datenblatt.pdf?10537896017176417241 (besucht am 01. 04. 2026).

[10] DFRobot. *DFRobotDFPlayerMini: Arduino Library for DFPlayer Mini MP3 Module*. 2026. URL: https://github.com/DFRobot/DFRobotDFPlayerMini (besucht am 26. 03. 2026).

[11] Dr. med. Andreas Busse. *Erste Hilfe bei Unterkühlung und Erfrierungen*. 2020. URL: https://www.rund-ums-baby.de/magazin/erste-hilfe-bei-unterkuehlung-und-erfrierungen (besucht am 01. 04. 2026).

[12] Espressif Systems. *Bluetooth API*. Espressif Systems. 2024. URL: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/bluetooth/index.html (besucht am 21. 03. 2026).

[13] Espressif Systems. *Bluetooth Low Energy Introduction*. Espressif Systems. 2024. URL: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/ble/get-started/ble-introduction.html (besucht am 07. 03. 2026).

[14] Espressif Systems. *Bluetooth Protocol Stack*. Espressif Systems. 2024. URL: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/bt-architecture/overview.html#bluetooth-protocol-stack (besucht am 08. 03. 2026).

[15] Espressif Systems. *ESP-IDF Programming Guide*. Espressif Systems. 2024. URL: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/get-started/index.html (besucht am 02. 04. 2026).

[16] Espressif Systems. *SMP*. Espressif Systems. 2024. URL: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/ble/smp.html (besucht am 08. 03. 2026).

[17] Mikiko Fukumoto u. a. „Studies of body movements during night sleep in infancy“. In: *Brain and Development* 3.1 (1981), S. 37–43. ISSN: 0387-7604. DOI: https://doi.org/10.1016/S0387-7604(81)80004-6. URL: https://www.sciencedirect.com/science/article/pii/S0387760481800046.

[18] YAGEO Nexensos GmbH. *M422 Pt-Temperature Sensor Datasheet*. Temperature coefficient TK = 3850 ppm/K. 2024. URL: https://cdn-reichelt.de/documents/datenblatt/D700/M_422_B_PT100_DB-DE.pdf.

[19] HengQing Electronics. *Mastering PWM Servo Motor Control*. HengQing Electronics. 2025. URL: https://heqingele.com/blog/mastering-pwm-servo-motor-control-2025/ (besucht am 23. 03. 2026).

[20] Henrik Blidh. *Bleak Documentation*. URL: https://bleak.readthedocs.io/en/latest/ (besucht am 21. 03. 2026).

[21] Jenoptik AG, Daniel Brenner. *Wärmebildkameras zum Fieber-Screening*. 2020. URL: https://www.jenoptik.de/news/blog/artikel/2020/06/18/waermebildkameras-zum-fieber-screening (besucht am 01. 04. 2026).

[22] Jonathan Hoffstadt, Preston Cothern. *Dear PyGui*. URL: https://dearpygui.readthedocs.io/en/latest/tutorials/first-steps.html (besucht am 21. 03. 2026).

[23] KiCad Development Team. *KiCad Documentation*. KiCad Development Team. 2024. URL: https://docs.kicad.org/ (besucht am 22. 03. 2026).

[24] MeinMed Redaktion. *Plötzlicher Kindstod (SIDS): Ursachen, Risikofaktoren und Prävention*. MeinMed. 2023. URL: https://www.meinmed.at/krankheit/ploetzlicher-kindstod/1603 (besucht am 07. 03. 2026).

[25] Nospampls. *SchedTask*. 2026-04-02. URL: https://github.com/Nospampls/SchedTask.

[26] Nugar Resistor Technology. *Keramikwiderstände: Merkmale und industrielle Anwendungen*. 2026-03-22. Nugar Resistor Technology. 2026. URL: https://nugar-resistor.com/de/resistencias-de-ceramica/.

[27] Ohmite Mfg Co. *Guide to Ceramic Resistors: Types, Functions and Applications*. 2026-03-22. 2024. URL: https://www.ohmite.com/guide-to-ceramic-resistors/.

[28] PlatformIO. *Development Platforms and Frameworks*. PlatformIO. 2024. URL: https://docs.platformio.org/en/latest/what-is-platformio.html (besucht am 02. 04. 2026).

[29] PlatformIO. *Development Platforms and Frameworks*. PlatformIO. 2024. URL: https://docs.platformio.org/en/latest/frameworks/index.html (besucht am 02. 04. 2026).

[30] Christian F. Poets und Gerhard Jorch. „Plötzlicher Kindstod“. In: *Pädiatrie: Grundlagen und Praxis*. Hrsg. von Georg F. Hoffmann u. a. Berlin, Heidelberg: Springer Berlin Heidelberg, 2020, S. 219–226. ISBN: 978-3-662-60300-0. DOI: 10.1007/978-3-662-60300-0_19. URL: https://doi.org/10.1007/978-3-662-60300-0_19.

[31] Real Python. *Python await Keyword*. URL: https://realpython.com/ref/keywords/await/ (besucht am 31. 03. 2026).

[32] Texas Instruments. *Logical Link Control and Adaptation Protocol (L2CAP)*. Texas Instruments. 2024. URL: https://software-dl.ti.com/lprf/sdg-latest/html/ble-stack-3.x/l2cap.html (besucht am 08. 03. 2026).

[33] Vogel Communications Group. *Was sind Servomotoren und wie funktionieren sie?* KEM Konstruktion Entwicklung Management. 2024. URL: https://kem.industrie.de/elektromotoren/was-sind-servomotoren-und-wie-funktionieren-sie/ (besucht am 23. 03. 2026).

[34] Wikipedia contributors. *Software framework*. 2025. URL: https://en.wikipedia.org/wiki/Software_framework (besucht am 02. 04. 2026).